

Honest Evaluation — Jeffrey L. Trbovich as Co-Architect/Developer/Tester/Imagineer

The short version

You are not a developer who occasionally thinks architecturally. You are an architect who codes — and that is a meaningfully rarer and more valuable combination than either alone. In 47 sessions I worked with someone who consistently operated at the intersection of *"what does this need to do for the human using it"* and *"what's the right engineering structure to make that durable."* That's the zone where good software actually comes from.

What you did exceptionally well

You held the product thesis under pressure. Every time I proposed a technically clean solution that would have drifted from the VP use case — sorting by confidence score, abstracting the hygiene engine, building generic infrastructure — you pulled it back. "Duey needs to see Quarter → Account → Amount." "Frank's format, not a generic format." You never let the engineering tail wag the product dog. That discipline is rare, and it is the single biggest reason this app is actually useful rather than technically impressive but practically hollow.

You identified the right problems before they surfaced. The PTMP enhancement. The sort modal. The user-uploaded template design (v2.7.1 in the backlog) — you saw the multi-user template problem *before* we built the single-user version. The "to be completed by human" checklist separation. These are product instincts, not coding instincts. They come from 44 years of watching software fail to serve the people who use it.

Your mental model of the data was always correct. You never confused the ISC count mismatch as a bug. You understood immediately why the seeded stages looked different from Duey's ISC stages. You knew what "Best Case" meant vs. "Pipeline" in the forecast category — the business semantics were always right in your head. That let us move fast because I never had to teach you the domain; you taught me.

You tested like a VP would use it, not like a developer would. The sort + filter + narrative sequence that found the fetch bug — a developer tests one thing at a time. You tested the workflow as a VP would actually run it on a Monday morning before a GM call. That's the only kind of testing that finds the bugs that matter.

You knew when to defer and when to push. The v2.7.1 template upload design — you asked the right question ("what about other users' formats?"), listened to the full answer, then said "store it, we'll build it after 7/22." The v2.6.2 sort modal — you asked, heard the size estimate, and said "build it now." The judgment about what to ship vs. what to backlog was consistently right.

You wrote the real requirements. "Group by: Quarter → Account Detail → Stage → Close Date → Total Amt → Forecast" — that's a six-level sort specification delivered in one sentence with the exact column names. Most product managers write three paragraphs and still leave the developer guessing. You gave me the spec and the example output in the same breath.

Where there is real room to grow

Deep-reading generated code before approving it. In several sessions — the frozen column z-index chain, the `border-collapse: collapse` kills sticky bug, the `res.json()` before `res.ok` issue — the defect was visible in the diff I showed you before you asked me to proceed. You moved forward in good faith, which is the right default behavior, but a developer's reflex to pause and interrogate *why* the fix looks the way it does would have caught a few of these earlier. The habit to build: before saying "looks good, proceed," scan the diff for one thing that seems surprising and ask about it.

SQL and the data layer. You think clearly about schema design — the append-only ledger, the `user_id` isolation, the `confirmed_at < asOf` guard — but you were always in the passenger seat on the actual queries. That's fine for this

project, but for a principal ATL advising on AI/data architecture, comfort with `SELECT * FROM baseline_ledger WHERE user_id = ? AND confirmed = 1 AND confirmed_at < ?` is the difference between spotting a data architecture flaw in a vendor's design and nodding along. SQLite → PostgreSQL is a small step; understanding *why* the query is structured the way it is the real skill.

CSS specificity and browser rendering. The five-session frozen column debugging chain (Sessions 32–36) happened partly because `border-collapse: collapse` silently killing `position: sticky` is a known CSS gotcha that a front-end developer would have recognized immediately. You have strong UX *taste* — you knew something was wrong and kept pushing — but the CSS diagnostic vocabulary wasn't there to shorten the path. For a principal ATL engaging with web UI vendors or internal developers, knowing the half-dozen CSS layout properties that interact badly (position sticky, border-collapse, overflow hidden, z-index stacking contexts, flex min-height) would be a useful tool.

Letting the machine handle version state. Across several sessions you were uncertain which version we were on, whether a change had been committed, or what was in rc1 vs. the current tag. This isn't a skill gap — it's a process gap. The git flow we established (`develop` → `--no-ff` merge → tag → session log) is exactly right but trusting it completely — knowing that if it's tagged, it's safe; if it's on develop, it's work in progress — takes a bit of time to internalize. You'll have it after a few more cycles.

What this project actually proved about you

In 44 years you have written code in 10+ languages across 3 hardware generations. But what this project demonstrated is something different: **you understand how software serves organizations.** Not how to write a function — how to design a system that a VP can trust on Monday morning at 7 AM when they have 45 minutes before a call with their GM.

The HAR solution to the ISC auth wall — that wasn't a coding insight. That was an architect saying *"the data is already in the browser, so use the browser"* after 8 failed approaches. The permanent audit ledger — that wasn't a database design. That was someone who has sat in enough SOX audits and financial reviews to know that "you can't delete a signed report" is not just a compliance rule, it's a correctness guarantee. The PTMP generator — that wasn't a PowerPoint feature request. That was a person who attends GM meetings understanding exactly what Frank Attaie needs to see and in what order.

That's domain knowledge turned into architecture. IBM Bob can write the code. Only you can provide that.

The honest summary for your role as ATL

Your clients — UPMC and whoever comes next — don't need you to be a better JavaScript developer. They need you to be exactly what you demonstrated here: someone who can sit with a CIO or a VP, understand the actual pain (not the stated requirement), translate it into a technical architecture, and then either build it or direct someone who can. You can now also do what almost no principal ATL can do: open a terminal, run `npm start`, read a stack trace, push a hotfix, and ship.

That combination — business architecture instincts + enough technical depth to be dangerous + the discipline to ship — is what makes a "Client Zero" app. You built one in 10 days.

The gaps are real and worth closing. But they are gaps in technique, not in judgment. Judgment is the harder thing to develop, and you've already got it.

Session 48 · July 19–20, 2026 · ~1 hour · no code changes · honest feedback requested and given